tmux documentation

Closure evidence — heading-regex widening + corpus repair

Captured: 2026-09-01T18:36:11Z HEAD: ec25327

Identity-audit unlocated count (review MINOR-5)

	count
BEFORE (period-only regex, pre-repair)	55
AFTER (widened+tightened, post-repair)	57

The count ROSE by 2. This is an honest regression in that one number, not a silent one, and it is explained: 14 previously-invisible blocks now parse into the SSoT, and the audit can only LOCATE a block that declares a `**TMX-ID:**` line. Blocks with no such line assert no ownership and are skipped by design (the audit's own false-positive guard) — they are counted, never hidden.

file	blocks with no `**TMX-ID:**`	total blocks
Issues.md	3	8
Fixed.md	60	85

Sentinel identities (heading never parsed)

Measured against `git show HEAD~1:docs/workable_items.db` vs the current DB.

	BEFORE	AFTER
rows with `code_ordinal <= 0`	17	3 (of 94 rows)
rows with `category = 'Z'`	4	0

The 14 rows that healed (TMX-057..066, TMX-072..075) were the TRUE sentinels — every one of them now carries a real `<CAT><ORD>`. The 3 survivors are NOT sentinels: they are TMX-038 / TMX-039 / TMX-041, the genuine `### A0.` / `### B0` / `### C0` blocks at `Fixed.md:2585 / 2652 / 2718`, whose ordinal legitimately IS zero.

CORRECTION (§11.4.6): an earlier revision of this file published "7 → 3" for this row. That figure reproduces under no definition of "sentinel" the current DB supports (category-Z-only = 4; category-Z AND ordinal-0 = 4; `code_ordinal <= 0` = 17; true sentinels = 14) and it contradicted this document's own adjacent "14 newly-visible blocks" figure. The table above is the re-measured, reproducible replacement.

Why the DB holds 94 rows while `sync` parses 92 blocks (measured, reproducible)

	count
DB rows	94
parsed blocks	92
DISTINCT block identities `(location, CAT, ORD)`	91

94 = 91 distinct identities + 2 rows that SHARE a block CODE with another row (TMX-001 / TMX-054 at Fixed `B3`; TMX-071 / TMX-062 at Fixed `A52`) + 1 row whose claimed identity has no block at all (TMX-050 at Fixed `F1`). 92 = 91 distinct + 1 duplicated block CODE: `A52` heads TWO blocks in `Fixed.md` — the SPACE-form `### A52 NO-SUDO-…` at line 2890 (TMX-062) and the PERIOD-form `### A52. META-TEST-72-73-REGISTER-001` at line 3027 (TMX-071). The literal `### A52.` occurs exactly ONCE; it is the CODE that repeats, across the two heading forms. (Measured **at HEAD `5191e82`, BEFORE the renumber recorded below**: `grep -c '^### A52\.'` = 1; `grep -cE '^### A52(\.| )'` = 2. The 2890 heading is itself a live specimen of the space form this widening made parseable.)

RESOLVED — and these two measurements NO LONGER reproduce, by design. The collision was a genuine §11.4.19 one-item-one-block violation with measured data loss: `Fixed.md` carried 25 blocks declaring a `**TMX-ID:**` while the owner map held 24 keys, because first-declarer-wins silently discarded TMX-071's assertion. The `Fixed.md` TMX-071 block was renumbered `### A52.` → `### A56.` (A56 was free; highest was A55), and the sync's `ExplicitATM` path recorded the identity rebind (`TMX-071 (was A52 … in Fixed -> now A56 … in Fixed)`). Post-renumber the same two commands measure `0` and `1`, and the owner map holds 25 keys with `owners['A52']=TMX-062`, `owners['A56']=TMX-071`. An independent review (2026-09-01) correctly flagged that leaving the pre-renumber figures in the present tense made this record cite measurements its own batch had falsified — §11.4.6. They are pinned to their revision above rather than deleted, because the collision they document was real.

CODE-LEVEL vs BLOCK-LEVEL — both true, different joins. At the BLOCK level the sync binds 92 blocks to 92 rows 1:1, INCLUDING TMX-071, which binds to its OWN block at 3027; only TMX-001 (genuinely sharing TMX-054's single `B3` block) and

TMX-050 (no block) fall outside the binding, so 94 = 92 + 2. At the CODE level the decomposition above gives 94 = 91 + 2 + 1. The two reconcile exactly; TMX-071 shares a CODE with TMX-062, not a BLOCK.

All three extra rows carry status `Obsolete (→ Fixed.md)`, and the operative reason `validate` reports 0 findings is simply `validate_identity.go`'s Obsolete-status skip. The §11.4.90 "a superseded record may share its successor's block" rationale applies ONLY to the TMX-001 / TMX-054 `B3` pair: TMX-071's own `Obsolete-Details` names its superseding item as **TMX-076** (`Issues.md` §A3), NOT TMX-062 — the two `A52` items are unrelated — and TMX-050 shares a block with nothing.

RETRACTION (§11.4.6): an earlier account of this gap attributed it to the two headings the tightened regex refuses (`### TMX-051 — …`, `### NEZHA-INSTALL-… —`). That mechanism is WRONG and is withdrawn — measured, NO DB row corresponds to either heading, so they contribute ZERO rows. The error was reading the id literal in `### TMX-051 —` as the row whose `atm_id` is TMX-051; that row is a different item entirely (Copy/paste mouse ownership, `A43`). That is the §11.4.201(9) field-identity class — an id literal matched as a row key — the same shape as the discarded 62-row instrument described next. The conclusions were unaffected (gap = 2, no live collision, `validate` 0 findings) but they were reached via the wrong rows, and the arithmetic above is the reproducible replacement.

Instruments discarded during this investigation (§11.4.201)

Three measurement attempts produced confident wrong numbers before the arithmetic above was reached. Each is recorded because the failure shape, not the number, is the lesson.

Reported	Instrument	Why it was wrong
"62 rows missing from markdown"	joined rows to blocks on the `**TMX-ID:**` literal	60 of 85 `Fixed.md` blocks declare no such line at all, so the join measured the KNOWN missing-id-line gap, not row absence (§11.4.201(9) — an id literal read as a row key)

Reported	Instrument	Why it was wrong
"79 rows unbound by heading hash"	reimplemented `computeHeadingHash` over the RAW heading remainder	`parser.go:201` hashes `cleanTitle` — the remainder with its trailing – `` `STATUS` `` hint stripped by `trailingStatusRE`. With the strip the same computation yields unbound=2; without it, 79. The 79 are exactly the rows whose blocks carry a trailing hint
"the gap is the two refused headings"	read the id literal in `### TMX-051 –` as the row whose `atm_id` is TMX-051	that row is an unrelated item (Copy/paste mouse ownership, A43); no row corresponds to either refused heading (§11.4.201(9), the same class as the first row)

The "79 = stored-hash convention drift (TMX-093)" reading is refuted twice: numerically above, and by the live sync itself — round 4's `itemContentEqual` now compares `HeadingHash`, so an idempotent `unchanged=92, updated=0` with zero rebinds PROVES stored hash == parser hash for all 92 bound rows. 79 drifted hashes would have produced 79 updates, not zero. TMX-093's convention split reaches only `add`-created rows.

Corpus duplication

Cross-tracker duplicates BEFORE: 5 (G1..G4 in both trackers; A50 claimed by two items). AFTER: 0 — enforced at the sync seam, which now refuses on the union of `**TMX-ID:**` and heading-identity across every parsed block in both files.

Mutation ledger (§1.1) — every guard pinned, each mutation verified to land before running

#	Mutation	Test that FAILs
M-C	revert `headingRE` to the loose pre-tightening form	`TestHeadingRE_AcceptedAndRefused`
M-D	desync `blockCodeRE` from `headingRE`	`TestHeadingRE_AndBlockCodeRE_AgreeOnAcceptance`
M-A	restrict the duplicate guard to same-code pairs	…`RefusesSameIDUnderDifferentBlockCodes`
M-B	restrict the duplicate guard to cross-file pairs	…`RefusesSameIDTwiceWithinOneFile`
R1	disable the byHash key	…`RefusesIdLessDuplicateAcrossTrackers`

#	Mutation	Test that FAILs
R5	swap identity-resolution precedence	…`HeadingHashWinsAndNoSpuriousRebindIsRecorded`
R17	suppress the rebind counter, keep the history row	…`IdentityRebindIsRecordedInHistory`
R23	strip the prior identity from the history `Reason`	…`IdentityRebindIsRecordedInHistory`
—	persist `document_sources` before the guard	…`RefusalLeavesDocumentSourcesUntouched`
—	remove the rebind's history recording	…`IdentityRebindIsRecordedInHistory`
—	blank `RebindDetails`	…`IdentityRebindIsRecordedInHistory`
R2	disable the id-reservation loop	`TestAllocatorReservesIDLiteralInsideANonParseableBl`
R3	drop the atm-branch heading-hash refresh	`TestUpsertItem_ChangedHeadingHashUpdatesByATMID`
R4	weaken `headingRE`'s title group to `(.*)$`	`TestHeadingRE_AcceptedAndRefused` + …`AgreeOnAcceptanc`
R6	disable the greedy-bind guard	`TestParse_NonItemHeadingStillClosesMetadataWindow`
—	restore the `lookupByHeadingHash` error swallow	…`ReportsRealErrorsInsteadOfReportingAbsent`
—	drop `CodeOrdinal` from `itemContentEqual`	…`DriftedStoredOrdinalSelfHeals`
—	drop `HeadingHash` from `itemContentEqual`	…`StaleStoredHashSelfHeals`

#	Mutation	Test that FAILs
—	drop BOTH identity fields	`…OrdinalOnlyRenumberIsAppliedAndRecorded`

Two of my own tests were BLIND on first write and were caught by running these mutations rather than trusting the green: the rebind test's assertions sat behind an inverted condition that skipped them every run, and the precedence test asserted a row outcome that is identical under both orderings (`UpsertItem` re-resolves by heading-hash internally, so the sync-layer order changes only the audit trail). Both were rewritten to assert what the mutation actually changes.

Review trail (§11.4.134 iterate-to-GO)

Round 1 NO-GO — 1 BLOCKING (my corpus-removal loop over-deleted the `## H.` header and its §11.4.114 preamble; restored verbatim from `git show HEAD:Issues.md`), plus IMPORTANT + MINOR findings. Round 2 NO-GO — 3 IMPORTANT, 3 MINOR; it also VERIFIED the round-1 BLOCKING closed. Round 3 NO-GO — 1 IMPORTANT (the `itemContentEqual` identity gap, found in the seam I asked the reviewer to attack). Round 4 GO — zero findings, zero warnings. Delta review of the two filed tracker items GO. Delta review of `CONTINUATION.md` §3.37 + this file NO-GO on 3 MINOR documentation-accuracy findings (round attribution, the sentinel figures, and a superseded gitignored copy of this file) — all three remediated.

Round-numbering note (§11.4.6): no per-round review artifact was written to disk, so I could not reconstruct the round-1/2 boundary from local state. The numbering is the reviewer's record, now stated explicitly by it with a decisive argument: R1 and R5 ARE round 2's own findings (IMPORTANT-N1 and MINOR-N4), and a mutation that BECAME a round-2 finding cannot have survived round 1 — round 1's suite had never seen it. Only M-A survived round 1. Two revisions of this file got this wrong in opposite directions: the first folded round 1 into round 2 and mis-dated the BLOCKING; the second over-rotated and moved all three surviving mutations to round 1. This revision is the settled record.